CTP : A Custom Transport Layer Protocol

Creator : RaspiestSheep3
Project Type: Transport Layer Protocol
Project Name : Compact Transfer Protocol (CTP)
Project Link : https://github.com/RaspiestSheep3/Custom_Networking_Protocol

1 - Summary

This RFC proposes a custom transport layer protocol I name Compact Transfer Protocol (CTP) as an alternative to existing protocols such as TCP, UDP and QUIC. This protocol is designed to be lossless and effective in extreme network conditions. An outline of design choices is given, and the protocol is simulated in C++.

2 - Problem Statement

The majority of modern applications rely on one of 3 transport layer protocols - TCP, UDP or QUIC. Whilst these protocols are undeniably effective, each face issues, especially when used in extreme network conditions:

- **TCP:** If a packet is lost, all subsequent pacers are paused despite their validity (Head-of-Line Blocking), which can heavily impact efficiency on slow networks. Moreover, if a packet is lost TCP assumes this is a congestion issue and slows down data transmission and therefore throughput.
- **UDP:** Because there is no insurance that all packets are being received on a network with high packet loss a large amount of packets will be dropped and never resent. This means key systems such as downloading files and messaging are unusable on extreme systems, and even applications such as image downloads become very low-quality.
- **QUIC:** Whilst it performs better than TCP and UDP on extreme networks, QUIC still faces some flaws. Primarily, QUIC encrypts the data before transmission, which can make it inefficient on the CPU for encrypting and decrypting data. QUIC is also based on UDP, which means that it is often blocked by firewalls as a byproduct of shutting down UDP-based malware attacks.

3 - Proposed Solution

3.1 - Project Proposal

The goal for this project is to design a custom transport-layer protocol able to form packets able to be transferred between machines. The protocol will then be implemented in C++ and tested by simulating key factors such as packet loss, latency and bandwidth.

3.2 - Design Specifications

The protocol should match some key specification points:

- Lossless data transmission (primary) - any data transmitted using the protocol should be guaranteed to arrive at the destination, even at extreme packet loss rate. This is the main flaw with UDP, and as such it should be corrected in this project
- High throughput under extreme conditions (primary) - the protocol should be efficient under extreme conditions, as this is the major flaw with TCP
- Not built on top of existing protocols (primary) - the protocol should not be layered over an existing transport layer protocol, as this is the major flaw with QUIC
- Lightweight (secondary) - the protocol should not be very intensive on the CPU or GPU, as this was a flaw with QUIC
- Easily implementable (secondary) - the protocol should be easy to implement within the standard OSI model

4 - Protocol Specification

5 - Protocol Testing

6 - Conclusion